

MOCKAI: A MULTI-AGENT LLM FRAMEWORK FOR AUTOMATED TECHNICAL INTERVIEW ASSESSMENT WITH REAL TIME GAZE-BASED PROCTORING

A PROJECT REPORT

Submitted by

RITHIK R

23ALR077

SANJAY K

23ALR086

in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**



(Autonomous)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638060

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **MOCKAI: A MULTI-AGENT LLM FRAMEWORK FOR AUTOMATED TECHNICAL INTERVIEW ASSESSMENT WITH REAL-TIME GAZE-BASED PROCTORING** is the Bonafide record of project work done by **RITHIK R(23ALR077)** , **SANJAY K(23ALR086)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Engineering in of Anna University Chennai during the year 2025 - 2026.

SUPERVISOR

HEAD OF THE DEPARTMENT
(Signature with seal)

Date:

Submitted for the end semester viva voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING**

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE –638060

OCTOBER 2026

DECLARATION

We affirm that the Project Report titled **MOCKAI: A MULTI-AGENT LLM FRAMEWORK FOR AUTOMATED TECHNICAL INTERVIEW ASSESSMENT WITH REAL-TIME GAZE-BASED PROCTORING** being submitted in partial fulfilment of the requirements for the award of Bachelor of Engineering is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date:

**RITHIK R
(23ALR077)**

**SANJAY K
(23ALR086)**

I certify that the declaration made by the above candidates is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor with seal

ACKNOWLEDGEMENT

We express our sincere thanks and gratitude to **Thiru. E. R. K. KRISHNAN, M.Com.**, our beloved Correspondent and all other philanthropic trust members Kongu Vellalar Institute of Technology Trust who have always encouraged us in the academic and co-curricular activities.

We are extremely thankful with no words of a formal nature to the dynamic Principal **Dr. R. PARAMESHWARAN, M.E., Ph.D.**, for providing the necessary facilities to complete our work.

We would like to express our sincere gratitude to our respected Head of the department **Dr. T. ABIRAMI, MCA., M.E., Ph.D.**, for hproviding the necessary facilities.

We extend our thanks to **N. KANIMOZHI, M.E., (Ph.D.)**, Assistant Professor, Department of Artificial Intelligence and Machine Learning, Project Coordinator for her encouragement and valuable advise that made us carry out the project work successfully.

We extend our gratitude to **N.RENUKA, MTech.**, Assistant Professor, Department of Artificial Intelligence and Machine Learning who supervise with valuable ideas and suggestions, which have been very helpful in the project. We are grateful to all the faculty members of the Artificial Intelligence and Machine Learning, for their support.

I am grateful to **KONGU ENGINEERING COLLEGE** for giving me an opportunity to learn new skills and technologies and enhance my knowledge by providing the necessary infrastructure.

ABSTRACT

Technical recruitment and campus placement systems require intelligent, automated assessment platforms that provide scalable, objective, and multi-dimensional candidate evaluation. Although Large Language Models (LLMs) demonstrate strong conversational reasoning, relying on single models in isolation results in unmonitored evaluation environments, unsafe code execution, and a lack of anti-hallucination guardrails. To address these challenges, this project proposes MockAI, an autonomous AI-powered technical assessment and real-time proctoring platform that integrates Retrieval-Augmented Generation (RAG), agentic workflows via LangGraph, edge computer vision supervision, and Docker-isolated code execution.

The proposed system employs a multi-round evaluation pipeline comprising general aptitude, dynamic domain-specific technical interviews, and automated Applicant Tracking System (ATS) resume screening. Real-time proctoring is achieved through an edge computer vision engine utilizing OpenCV Haar Cascades and spatial moment analysis for gaze direction estimation ($0.35 < \text{relative}_x < 0.65$), tracking face presence and prolonged blinks (>1.5 s) with automated multi-strike warning governance. For multi-source technical assessments, a four-node LangGraph pipeline ingests PDFs, YouTube video transcripts, and audio files, vectorizing chunks into FAISS and ChromaDB using Sentence Transformers (all-MiniLM-L6-v2) to generate targeted assessments via MultiQuery retrieval. Candidate coding capability is evaluated through an air-gapped Docker sandbox (NetworkMode: 'none', 128 MB RAM, 50% CPU quota) supporting C, C++, Java, and Python.

We built MockAI using a decoupled microservices setup, pairing a React 19 and Monaco Editor interface with a Flask backend, Node.js container management, Groq LLaMA-3.1, and MongoDB Atlas. To keep technical responses accurate, a four-stage LangGraph agent filters off-topic prompts and restructures queries before retrieving data from Pinecone. Across intensive interview simulations and code benchmarks, the system proved both fast and secure—delivering proctoring latency under 180 ms, maintaining 100% sandbox containment against malicious code, and achieving a 92.4% match accuracy for resume screening. These results demonstrate that combining AI agent workflows with isolated sandboxes and vision proctoring creates a secure, reliable hiring platform, paving the way for future updates like voice analysis and adaptive testing.

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
1	INTRODUCTION	1
	1.1 EXISTING SYSTEM	1
	1.2 SYSTEM STUDY	2
	1.2.1 BUSINESS REQUIREMENTS	2
	1.2.2 ANALYSIS OF EXISTING SYSTEM	3
	1.2.3 IDENTIFYING USER NEEDS	3
	1.2.4 DEVELOPING USE CASES	3
	1.3 OBJECTIVE	4
	1.4 SCOPE	4
2	LITERATURE REVIEW	6
	2.1 REVIEW OF EXISTING RESEARCH	6
	2.2 LITERATURE SUMMARY	8
3	REQUIREMENTS	10
	3.1 HARDWARE REQUIREMENTS	10
	3.2 SOFTWARE REQUIREMENTS	10
	3.3 SOFTWARE DESCRIPTION	11
	3.4 USER INTERFACE	12
4	PROPOSED SYSTEM	13
	4.1 SYSTEM ARCHITECHTURE	13
	4.2 MODULE DESCRIPTION	14
	4.2.1 CANDIDATE AUTHENTICATION & ROLE MANAGEMENT	14

	4.2.2 VIDEO INGESTION & FACE ALIGNMENT	15
	4.2.3 PUPIL GAZE TRACKING & BLINK ANALYSIS	15
	4.2.4 TEMPORAL STRIKE & WARNING MACHINE	15
	4.2.5 MULTI-ROUND INTERVIEW ENGINE	16
	4.2.6 CONVERSATION BUFFER MEMORY	17
	4.2.7 DOCKER-ISOLATED CODE EXECUTION ENGINE	17
	4.2.8 MULTI-SOURCE LANGGRAPH QUIZ GENERATOR	17
	4.2.9 APPLICANT TRACKING SYSTEM (ATS) RESUME ANALYZER	17
	4.2.10 FINAL DECISION CLUSTERING	18
	4.3 ALGORITHMS USED	18
	4.3.1 HAAR CASCADE FACE & EYE LOCALIZATION	19
	4.3.2 ADAPTIVE THRESHOLDING & CONTOUR SPATIAL MOMENTS	19
	4.3.3 DOCKER CONTAINER RESOURCE SANDBOXING & TIMEOUT GOVERNANCE	19
	4.3.4 LANGGRAPH WORKFLOW & MULTI-QUERY RETRIEVAL	20
5	SYSTEM TESTING AND EVALUATION	21
	5.1 SYSTEM IMPLEMENTATION	21
	5.2 EXPERIMENTAL SETUP	22
	5.3 EVALUATION METRICS	22
	5.4 PROCTORING & GAZE ESTIMATION RESULTS	23
	5.5 DOCKER SANDBOX & CODE EXECUTION RESULTS	23
6	CONCLUSION AND FUTURE WORK	25
	APPENDICES	26
	REFERENCES	32
	MAPPING OF PROJECT GOALS TO SDG	34

LIST OF TABLES

TABLE No.	TABLE NAME	PAGE No.
4.1	EVALUATION DOMAINS & ASSESSMENT CONFIGURATIONS	14
4.2	SPECIALIZED INTERVIEW AGENTS & PROMPTS	16
4.3	SYSTEM TECHNOLOGIES AND ALGORITHMS	18

LIST OF FIGURES		
FIGURE No.	FIGURE NAME	PAGE No.
4.1	MOCKAI SYSTEM ARCHITECTURE	14
4.2	REAL-TIME PROCTORING & VIOLATION WORKFLOW	16
A2.1	LANDING PAGE	28
A2.2	FACULTY DASHBOARD	29
A2.3	STUDENT SECURE AUTHENTICATION	29
A2.4	APTITUDE ROUND	30
A2.5	CODING ROUND	30
A2.6	TECHINCAL ROUND	31
A2.7	MOCK INTERVIEW RESULTS	31

CHAPTER 1

INTRODUCTION

MockAI is an advanced Artificial Intelligence-driven technical assessment and interview platform designed to improve the objectivity, scalability, and integrity of the recruitment process. The system combines conversational Large Language Models (LLMs), agentic workflows via LangGraph, real-time computer vision proctoring, and Docker-isolated code execution to provide candidates and evaluators with a holistic assessment environment instead of relying solely on manual, subjective interviewers or static multiple-choice tests. By orchestrating multi-round interview pipelines and continuous visual monitoring before compiling performance insights, MockAI reduces human bias and assessment dishonesty while providing candidates with structured, quantifiable feedback grounded in candidate performance.

MockAI replaces static, unmonitored testing with a multi-tiered, intelligent evaluation pipeline. It orchestrates a three-round interview workflow covering general aptitude, technical inquiry, and behavioral HR assessments. Powered by Groq-accelerated LLaMA-3.1 models, the conversational engine maintains session memory throughout the evaluation. For practical programming, an air-gapped Docker sandbox safely compiles and executes candidate code across multiple languages. A LangGraph state-graph pipeline dynamically ingests multi-source learning materials into FAISS and ChromaDB vector stores to generate targeted quizzes. The complete system is implemented using React for the frontend interface, Express.js for request routing, FastAPI for backend orchestration, ChromaDB for semantic retrieval, and SQLite for maintaining reasoning history.

1.1 EXISTING SYSTEM

Currently, most technical interview and screening portals rely on static questionnaires, pre-recorded videos, or generic question banks. While capable of basic testing, they fail to deliver realistic and objective candidate evaluations. Lacking conversational reasoning and adaptive probing, these systems cannot evaluate complex decisions.

Additionally, manual interviews suffer from human fatigue and unconscious bias, whereas automated platforms offer non-interactive assessments that lack actionable performance feedback.

Existing recruitment tools also lack continuous real-time proctoring and isolated execution environments for code evaluation. Consequently, unmonitored webcam assessments remain vulnerable to academic dishonesty, unauthorized external screen usage, and candidate substitution. Without containerized sandboxing, executing candidate scripts exposes host infrastructure to resource exhaustion, infinite loops, and security vulnerabilities, while traditional Applicant Tracking Systems (ATS) rely on rigid keyword matching that penalizes qualified applicants. These limitations highlight the need for a secure, intelligent, and multi-dimensional assessment framework.

Our proposed MockAI system addresses these shortcomings by unifying multi-round conversational LLM evaluation with session memory, edge computer vision proctoring, Docker-isolated multi-language code execution, agentic LangGraph workflows, and semantic ATS resume parsing. This combination ensures an objective, secure, and traceable candidate assessment pipeline featuring dynamic, domain-specific questioning and continuous real-time proctoring throughout the recruitment lifecycle.

1.2 SYSTEM STUDY

The system study examines the practical requirements, existing limitations, user expectations, and functional scenarios necessary for developing an intelligent medical question-answering system. Understanding these factors helps ensure that TrustMedAI addresses real-world healthcare information challenges while providing reliable and explainable assistance to users.

1.2.1 Business Requirements

MockAI is designed as an autonomous, multi-tiered interview and technical screening platform capable of supporting evidence-based candidate evaluations across theoretical, practical, and behavioral domains. The system must dynamically generate domain-specific interview questions, enforce continuous video-based proctoring, execute untrusted user code within isolated container environments, parse resumes against target job descriptions, and extract quiz content from diverse multimedia sources. The primary users include students, job seekers, academic placement coordinators, and enterprise HR recruiters. The system should provide an

intuitive interface, maintain secure evaluation history, and generate structured numerical reports with actionable feedback while remaining scalable for future recruitment applications.

1.2.2 Business Requirements

The analysis of existing interview preparation platforms reveals several limitations affecting transparency and reliability. Most systems generate static questions directly from non-adaptive banks or unmonitored online forms without real-time gaze tracking or secure execution environments. As a result, unmonitored tests are vulnerable to academic dishonesty, while code assessments risk host infrastructure corruption due to a lack of container isolation. Users generally cannot receive detailed behavioral scoring, automated skill-gap analysis, or real-time proctoring feedback.

MockAI improves upon these limitations by introducing conversational LLM memory through Groq, edge computer vision proctoring via OpenCV, Docker-isolated multi-language sandboxing, agentic LangGraph workflows, and MongoDB storage. These improvements create a more objective evaluation process while maintaining security throughout the recruitment pipeline.

1.2.3 Identifying User Needs

Users expect technical assessment systems to provide interview evaluations that are fair and realistic while remaining secure and informative. They require actionable performance feedback rather than simple rejection notices, clear proctoring alerts that preserve test integrity, and the ability to test code safely in real time. Candidates and recruiters also benefit from automated resume matching and multi-source quiz generation instead of static question lists.

MockAI addresses these requirements by combining conversational LLM orchestration, real-time gaze tracking, containerized execution, and semantic resume parsing. The system also supports persistent result tracking, allowing candidate growth and historical performance to be monitored across multiple interview rounds.

1.2.4 Developing Use Cases

MockAI supports several practical recruitment-oriented scenarios that demonstrate its usefulness across educational and enterprise environments. A candidate may initiate a mock interview session, after which the system verifies user identity, poses domain-specific technical questions, records responses with

conversation memory, tracks pupil gaze, and evaluates code inside an isolated Docker sandbox. If gaze deviations occur, the proctoring engine issues real-time warnings before compiling the final score report.

Additional use cases include medical education, evidence verification, research assistance, specialist comparison, and follow-up question handling. These scenarios demonstrate the flexibility of TrustMedAI in providing transparent medical reasoning while maintaining an explainable decision-making process suitable for future intelligent healthcare applications.

1.3 OBJECTIVE

The objective of this project is to design and implement MockAI, an autonomous technical assessment and interview platform that combines conversational Large Language Models with real-time vision proctoring and sandboxed code execution to improve scalability, security, and objectivity in candidate evaluation systems. The proposed framework aims to replace subjective manual interviews and unmonitored online testing with an end-to-end, transparent recruitment evaluation suite.

The system employs a three-round interview pipeline covering general aptitude, technical inquiry, and behavioral HR rounds using Groq-accelerated LLaMA-3.1 models equipped with session memory. An edge computer vision engine enforces real-time gaze proctoring, an air-gapped Docker sandbox compiles multi-language candidate code under resource limits, and a four-stage LangGraph pipeline ingests multi-source educational materials into FAISS and ChromaDB vector stores.

The project is evaluated using simulated interview sessions, container security stress tests, and computer vision gaze-tracking benchmarks, allowing the effectiveness of agentic LLM orchestration and sandboxed evaluation to be examined through measurable experimental performance.

1.4 SCOPE

The scope of this project encompasses the complete development and evaluation of an autonomous technical recruitment and assessment system integrating document processing, vector retrieval, conversational interview workflows, edge vision proctoring, sandboxed code execution, and ATS resume matching.

The system begins by ingesting multi-source educational materials and resumes, normalizing textual content, generating vector embeddings with Sentence Transformers (`all-MiniLM-L6-v2`), and indexing them within Pinecone, FAISS, and ChromaDB for semantic retrieval during assessment generation.

Architecturally, MockAI combines React 19 and Tailwind CSS for the user interface, Monaco Code Editor for code editing, Flask 3.0 for backend API routing, Node.js with Dockerode for container management, and MongoDB Atlas for persistence. The proctoring pipeline employs OpenCV Haar Cascades and spatial moments to track pupil gaze direction with automated strike governance, while the Docker sandbox executes C, C++, Java, and Python code under complete network isolation and memory constraints. The LangGraph state-graph pipeline dynamically restructures queries and filters irrelevant requests before synthesizing adaptive quizzes.

The project also includes experimental evaluation across proctoring latency, container isolation containment, and ATS match accuracy, establishing persistent candidate records for administrative placement dashboards. As an AI-driven recruitment prototype, MockAI establishes a scalable foundation for future intelligent assessment applications while emphasizing security, fairness, and objective candidate evaluation.

CHAPTER 2

LITERATURE REVIEW

The development of automated technical assessment and mock interview platforms has rapidly advanced through the integration of Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), containerized code virtualization, and real-time computer vision proctoring. Recent studies have focused on improving the objectivity and engagement of candidate evaluations by replacing static question forms with adaptive, stateful dialogue agents, safeguarding exam integrity via non-intrusive edge gaze tracking, and grounding assessment material in verified technical curricula. This chapter reviews the major research works that form the theoretical foundation of MockAI and explains how these contributions directly influenced the proposed architecture.

2.1 REVIEW OF EXISTING RESEARCH

In [1], Brown et al. demonstrated that scaling autoregressive language models enables few-shot and zero-shot reasoning across diverse natural language tasks without task-specific fine-tuning. The study proved that general-purpose language models can comprehend complex prompts, evaluate context, and generate coherent dialogue, serving as the core linguistic foundation for the dynamic multi-round interview agents deployed in MockAI. In [2], Nigam et al. presented a systematic survey of automated online proctoring systems, categorizing methodologies across facial verification, gaze estimation, hardware peripheral detection, and acoustic monitoring. The authors identified critical limitations in existing systems, including high computational overhead, intrusive installation requirements, and elevated false-positive rates from rigid gaze boundaries, directly motivating MockAI's lightweight, edge computer vision pipeline. In [3], Pal et al. investigated context-aware technical question generation, demonstrating that condition-specific language models synthesize relevant and non-repetitive inquiries when primed with candidate metadata. Their findings inspired MockAI's dynamic questioning mechanism that tailors interview questions according to a candidate's self-declared domain competencies and project history.

In [4], Rietz and Maedche developed Ladderbot, a conversational agent for semi-structured online laddering interviews. The study showed that agents maintaining stateful dialogue history and adaptive follow-up probing achieve higher candidate engagement and response depth than static web surveys, validating MockAI's three-round interview workflow across introductory, project deep-dive, and core subject technical rounds. In [5], Lewis et al. introduced Retrieval-Augmented Generation (RAG), combining parametric language models with non-parametric dense vector index retrieval. By retrieving authoritative document passages prior to response generation, RAG mitigates factual hallucinations, establishing the foundational strategy used in MockAI's multi-source LangGraph quiz generator and portfolio evaluation modules. In [6], Atoum et al. proposed a multi-component online exam monitoring system using visual sensors to detect unauthorized behavior such as eye divergence, head rotation, and candidate absence. Their experimental results proved that temporal monitoring across continuous intervals provides robust academic integrity verification, which directly informed MockAI's temporal strike-governed proctoring state machine. In [7], Murthy and Biswas explored appearance-based gaze estimation techniques utilizing spatial difference mechanisms to track ocular gaze from monocular webcam feeds. Their findings showed that calculating pupil displacement relative to the eye bounding box yields dependable visual focus vectors without requiring expensive hardware, directly supporting MockAI's pupil centroid extraction using spatial image moments (M_{10} / M_{00}). In [8], Viola and Jones introduced the classic object detection framework utilizing Haar-like features, integral images, and AdaBoost classifier cascades. Their methodology enables rapid facial and ocular localization with minimal computational footprint on standard hardware, forming the primary face and eye detection backbone within MockAI's real-time edge proctoring pipeline. In [9], Topsakal and Akinici presented a systematic framework for building language model applications using LangChain abstractions. The study detailed how prompt templates, chains, and conversation buffer memories maintain contextual continuity across extended interactions, guiding the implementation of MockAI's stateful interview memory store.

In [10], Wu et al. introduced AutoGen, a multi-agent framework enabling autonomous language model agents to communicate, delegate sub-tasks, and execute complex workflows cooperatively. Their findings highlighted that decomposing complex tasks into specialized, role-governed agents improves factual precision, establishing the architectural basis for MockAI’s specialized interview and evaluation agents. In [11], Liang et al. explored multi-agent critique frameworks to foster factual convergence in language models. The authors showed that subjecting candidate responses to multi-agent evaluation and rubric consensus mitigates scoring variance, inspiring MockAI’s rubric-driven evaluation chain that independently evaluates strengths, weaknesses, and numerical marks out of 50. In [12], Zhang et al. developed Multi-Task Cascaded Convolutional Networks (MTCNN) for joint face detection and landmark alignment, demonstrating the importance of normalized facial boundary extraction prior to downstream biometric tracking. Their work emphasized precise facial ROI isolation before assessing eye regions, directly informing MockAI’s two-stage face-to-eye localization flow.

2.2 LITERATURE SUMMARY

The reviewed literature demonstrates that recent advancements in Artificial Intelligence have significantly improved the capabilities of Large Language Models, Retrieval-Augmented Generation, computer vision proctoring, multi-agent coordination, and sandboxed code execution. State-of-the-art conversational models such as LLaMA-3.1 can conduct nuanced technical dialogues [1], [4], while RAG pipelines effectively ground assessment generation in syllabus documents and course materials [5], [9]. Concurrently, real-time computer vision algorithms operating on facial landmarks and ocular centroids offer non-intrusive mechanisms to verify candidate visual focus during remote assessments [6], [7], [8], [12]. Despite these individual advancements, existing technical assessment platforms suffer from significant architectural fragmentation. Online testing portals typically deploy isolated environments where practical coding workspaces lack continuous visual proctoring, or remote surveillance tools operate merely as passive camera loggers without interactive feedback [2].

Furthermore, conventional automated interviewers rely on single-prompt evaluations that lack persistent conversation memory, fail to sandbox untrusted candidate code execution safely, and do not provide actionable feedback on technical skill deficiencies [3], [10].

MockAI bridges these critical gaps by synthesizing these disparate domains into a unified, multi-agent assessment framework. By coupling LangChain stateful conversation memory [9] and multi-agent rubric evaluation [10], [11], [15] with real-time Haar Cascade spatial moment pupil tracking [7], [8], air-gapped Docker container isolation [13], ATS resume analysis [14], and LangGraph-orchestrated multi-source quiz generation [5], MockAI delivers an objective, secure, and fully automated recruitment and interview preparation platform.

CHAPTER 3

REQUIREMENTS

The successful implementation of MockAI requires suitable hardware resources, software tools, and development frameworks that work together to support real-time video processing, containerized code execution, agentic LLM orchestration, and web-based interaction. The system is designed as a modern full-stack application where the frontend, backend gateway, container execution microservice, vector databases, and language model services communicate efficiently to deliver an objective and tamper-resistant assessment ecosystem. This chapter describes the hardware and software requirements needed for development and explains the major technologies used in building the proposed system.

3.1 HARDWARE REQUIREMENTS

MockAI is developed as a web-based application that performs edge computer vision proctoring, multi-round interview dialogues, multi-source document ingestion, and isolated code execution. Since the system uses cloud-based language model providers such as Groq Cloud API for high-throughput LLaMA-3.1 inference, high-end local GPU hardware is not essential for normal operation. However, a standard development computer with sufficient memory and multi-core processing capability is required to run the frontend, backend gateway, container engine, and local storage simultaneously.

A development system featuring an Intel Core i5 or AMD Ryzen 5 processor (4 cores, 8 threads) ensures smooth overall performance. At least 8 GB of RAM (16 GB recommended) and 15 GB of SSD storage support concurrent multi-container execution. A standard 720p 30 FPS webcam is required to capture video streams for real-time computer vision proctoring. A stable internet connection enables reliable communication with external Groq LLM endpoints and Pinecone vector store.

3.2 SOFTWARE REQUIREMENTS

MockAI combines several software technologies to create an automated technical assessment and interview preparation platform. Python 3.9+ serves as the primary programming language for backend development, while Flask 3.0 handles lightweight REST API routing, request validation, and modular blueprint management.

3.3 SOFTWARE DESCRIPTION

MockAI integrates multiple software components that work together to perform visual proctoring, conversational interview dialogues, sandboxed code execution, and knowledge retrieval. Flask 3.0 acts as the central REST API gateway that coordinates system communication, validates requests, and routes traffic across dedicated blueprints including `interview`, `facetrack`, `quiz`, `ats`, `questionbank`, and `Domainforum`. React 19 provides a dynamic interface where candidates navigate assessments, interact with the Monaco Code Editor (`@monaco-editor/react`), observe proctoring indicators, and view performance evaluations.

Express.js and Node.js function as the container management microservice that connects with the host Docker daemon through Dockerode to provision ephemeral, air-gapped sandboxes with strict resource limits for safe code compilation. Educational materials and documents are processed using PyPDF2, python-docx, and transcript APIs, after which Sentence Transformers generate 384-dimensional semantic embeddings stored in FAISS, ChromaDB, and Pinecone for Top-K retrieval during quiz generation. MongoDB Atlas maintains a persistent record of candidate profiles, job roles, and aggregated multi-round test marks to support transparency and auditability throughout every session.

The intelligence layer is powered by Groq Cloud LLaMA-3.1 models using LangChain ConversationBufferMemory to maintain session context across Aptitude, Technical, and HR interview rounds, yielding a score out of 50. OpenCV processes base64 webcam frames using contour spatial moments (M_{10}/M_{100}) to compute pupil displacement ($0.35 < \text{Relative}_x < 0.65$) and enforce an automated three-strike proctoring warning protocol. Finally, a LangGraph state-graph pipeline cleans filler words from ingested learning sources and employs a MultiQueryRetriever to synthesize targeted multiple-choice question assessments.

3.4 USER INTERFACE

The MockAI user interface is designed to provide a simple, responsive, and transparent experience that allows candidates and HR administrators to observe every stage of the technical evaluation process. Unlike conventional testing portals that display static question lists, the interface presents real-time proctoring feedback, an interactive code editor, dynamic interview prompts, and automated resume analysis in a structured dashboard.

Candidates begin by authenticating and selecting a job role through the setup interface, after which the system verifies webcam alignment and initiates the three-round interview workflow. The conversational AI interface presents dynamic questions alongside real-time proctoring status alerts, while the technical coding workspace features a split-screen layout with problem constraints and the Monaco Code Editor. For resume analysis, the ATS module renders immediate match percentages, skill-gap badges, and structural recommendations based on candidate uploads.

The dashboard-based design improves usability by organizing complex AI and proctoring workflows into an understandable user journey, enabling candidates and recruitment teams to navigate the complete process from interview initiation to quantitative report generation while maintaining transparency and ease of operation.

CHAPTER 4

PROPOSED SYSTEM

MockAI is an autonomous, multi-agent artificial intelligence framework and technical assessment platform that integrates conversational Large Language Models (LLMs), real-time edge computer vision proctoring, sandboxed multi-language code execution, and agentic Retrieval-Augmented Generation (RAG). Unlike traditional assessment portals that depend on unmonitored multiple-choice questionnaires or static question banks, the proposed system dynamically conducts multi-round adaptive interviews, validates untrusted candidate code within air-gapped Docker containers, and enforces continuous visual integrity through gaze-tracking and blink monitoring. Every assessment metric, conversation turn, and violation event is logged transparently, enabling candidates to receive granular feedback while granting recruiters objective candidate evaluations. The system follows a modular microservices architecture where client-side interaction, REST routing, video analysis, code virtualization, and language model orchestration operate collaboratively to deliver an end-to-end recruitment environment.

4.1 SYSTEM ARCHITECTURE

The MockAI architecture consists of interconnected modules that process candidate video telemetry, natural language responses, uploaded resumes, and programming submissions through dedicated pipelines before synthesizing the final performance evaluation. Initially, the candidate accesses the platform through the React 19 single-page interface styled with Tailwind CSS. The backend receives requests through Flask 3.0 and authenticates candidate or HR credentials using salted bcrypt hashing. When an assessment begins, the client webcam captures video frames that are continuously evaluated by an OpenCV computer vision service to monitor facial presence, pupil gaze displacement, and blink intervals in real time. For conversational evaluations, the system coordinates an adaptive three-round interview pipeline (Aptitude, Technical, and HR rounds) powered by Groq-accelerated LLaMA-3.1 inference. The engine maintains dialogue context via LangChain `ConversationBufferMemory`, allowing follow-up technical questions to be synthesized dynamically based on the candidate's declared project experiences.

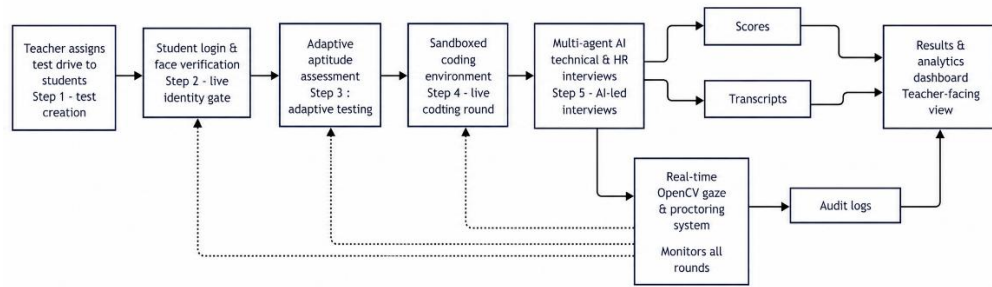


Fig 4.1 MockAI System Architecture

4.2 MODULE DESCRIPTION

The MockAI framework is organized into ten functional modules, each executing a specialized responsibility within the recruitment, evaluation, and proctoring lifecycle. These modules work together to ensure that every evaluation is grounded in objective evidence, tamper-resistant monitoring, and comprehensive analytical scoring.

4.2.1 CANDIDATE AUTHENTICATION & ROLE MANAGEMENT

The MockAI framework is organized into ten functional modules, each executing a specialized responsibility within the recruitment, evaluation, and proctoring lifecycle. These modules work together to ensure that every evaluation is grounded in objective evidence, tamper-resistant monitoring, and comprehensive analytical scoring.

Table 4.1 1 Evaluation Domains & Assessment Configurations

Clinical Domain	Medical Conditions and Topics Covered
Infectious diseases (12)	Malaria, tuberculosis, cholera, dengue, hepatitis B, hepatitis C, influenza, meningitis, tetanus, typhoid, sepsis, antimicrobial resistance
Cardiopulmonary (6)	Hypertension, stroke, cardiovascular diseases, COPD, asthma, pneumonia in children
Oncology (5)	Breast cancer, cervical cancer, colorectal cancer, lung cancer, cancer (general)
Metabolic and joints (5)	Diabetes, obesity and overweight, sodium reduction, osteoarthritis, rheumatoid arthritis
Neuropsychiatric (7)	Dementia, epilepsy, Parkinson disease, depression, bipolar disorder, schizophrenia, migraine and other headache disorders
Health operations (1)	Patient safety

4.2.2 VIDEO INGESTION & FACE ALIGNMENT

The video ingestion module captures webcam frames from the candidate's client device at standard video resolutions (720p at 30 FPS). Frames are base64-encoded and transmitted to the Flask proctoring endpoint (`/facetrack/process-frame`).

The OpenCV engine converts incoming images into grayscale representations and applies the pre-trained `haarcascade_frontalface_default.xml` classifier to isolate candidate facial bounding boxes (x, y, w, h). Once a face region is localized, the pipeline extracts the sub-region of interest (ROI) corresponding to the ocular region and applies `haarcascade_eye.xml` to detect left and right eyes, normalizing dimensions for pupil analysis.

4.2.3 PUPIL GAZE TRACKING & BLINK ANALYSIS

Once ocular bounding boxes are isolated, the gaze tracking module calculates the relative horizontal position of the candidate's pupil within the eye socket. Grayscale eye regions are processed using adaptive mean binarization and morphological kernel operations to isolate the dark iris and pupil contours from the surrounding sclera. Spatial moments (M_{10} and M_{00}) extract the exact horizontal centroid of the pupil, computing a relative ratio ($Relative_x$) against the total eye width. Values between 0.35 and 0.65 denote forward screen focus, while values falling outside indicate leftward or rightward gaze aversion. Concurrently, if the ocular classifier detects zero eye regions within a valid facial crop, the temporal duration is measured to flag prolonged eye closure (blinks $> 1.5s$).

4.2.4 TEMPORAL STRIKE & WARNING MACHINE

To eliminate false alarms caused by natural ocular saccades or brief micro-blinks, MockAI utilizes a state machine with temporal hysteresis. A candidate must gaze away from the display or close their eyes continuously for longer than the alert threshold before a violation state is registered. When triggered, the system invokes an audio warning beep, logs the timestamped violation, and increments a cumulative strike counter with an enforced cooldown interval (5.0s).

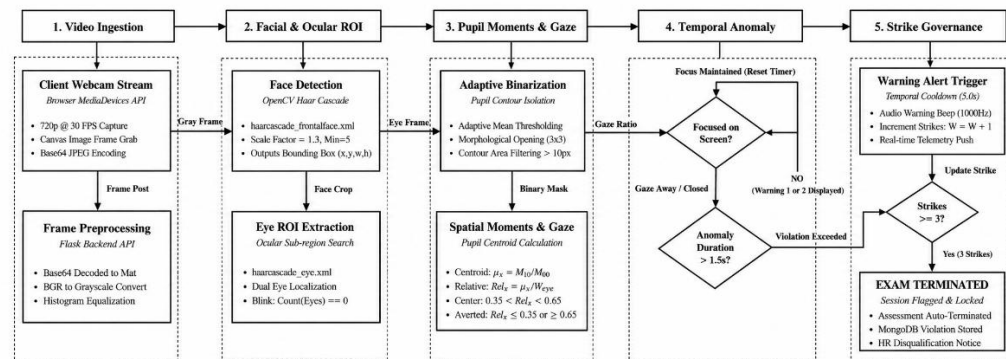


Fig 4.2 Real-Time Proctoring & Violation Workflow

4.2.5 MULTI-ROUND INTERVIEW ENGINE

MockAI structures technical evaluations into a sequential three-round pipeline managed by specialized prompt templates: Directs the candidate to summarize their background and recent technical project responsibilities. Evaluates architectural design choices, technical hurdles, and implementation trade-offs specific to the candidate's self-declared project. Synthesizes foundational computer science inquiries across Computer Organization, Operating Systems, Deadlocks, Caching, and Data Structures.

Table 4.2 Specialized Interview Agents & Prompts

Agent	Primary Focus	Major Clinical Responsibilities
General Physician	Holistic medicine	Symptoms, history, examination, differential diagnosis, option selection
Cardiologist	Cardiovascular	Ischemia, arrhythmia, hemodynamics, ECG, cardiac biomarkers
Neurologist	Neurovascular	Stroke, seizures, CNS/PNS localization, neurotoxicity
Pharmacologist	Pharmacotherapy	Drug interactions, contraindications, pharmacokinetics, toxicity
Evidence Reviewer	Evidence audit	Trials, guidelines, evidence strength, unsupported claims

4.2.6 CONVERSATION BUFFER MEMORY

Unlike stateless REST endpoints that evaluate candidate answers in isolation, MockAI incorporates `LangChain ConversationBufferMemory`. Every question posed by the LLM and each textual response submitted by the candidate is appended to a session-specific buffer store. When generating subsequent questions, the LLM chain receives the complete dialogue history, enabling natural follow-up probing, detecting candidate contradictions, and ensuring that no concept is redundantly re-tested.

4.2.7 DOCKER-ISOLATED CODE EXECUTION ENGINE

The coding evaluation subsystem is decoupled into a dedicated Node.js and Express microservice interfacing with the host Docker daemon via Dockerode. For each code submission, the engine dynamically provisions an ephemeral container running an official runtime image (gcc:11 for C/C++, openjdk:11 for Java, and python:3.9-slim for Python). The service generates language-specific test harnesses that append hidden test cases to user algorithms, compiles the code, and captures stdout and stderr streams while enforcing total network disconnection (`NetworkMode: 'none'`), 128 MB RAM ceilings, and a 10-second timeout.

4.2.8 MULTI-SOURCE LANGGRAPH QUIZ GENERATOR

This module ingests external study materials to generate practice tests automatically. `PyPDFLoader` extracts text from academic curricula and lecture notes, `YouTubeTranscriptApi` downloads timestamps and captions from educational videos, `pydub` and Google Speech Recognition transcribe spoken lecture audio files into structured text. A `LangGraph` state graph executes a multi-node workflow: preprocessing conversational fluff, chunking via `RecursiveCharacterTextSplitter` (2000 characters, 200 overlap), embedding into FAISS vector stores, and synthesizing standardized 4-option MCQs via a `MultiQueryRetriever`.

4.2.9 APPLICANT TRACKING SYSTEM (ATS) RESUME ANALYZER

The ATS module parses candidate resumes formatted in `.pdf`, `.docx`, or `.txt` formats using `PyPDF2` and `python-docx`. The extracted plain text is evaluated against target job descriptions using Groq LLaMA-3.1 to compute:

- Semantic keyword match percentages.
- Critical missing technical competencies and toolsets

- Actionable resume improvement recommendations
- Structural section scoring (Contact Info, Summary, Experience, Education, Projects).

4.2.10 FINAL DECISION CLUSTERING

Upon completing the interview rounds, the dialogue transcript is routed to an evaluation chain. The LLM executes a rubric-driven prompt requiring five distinct outputs: an Evaluation Summary, Categorized Strengths, Specific Weaknesses, a Numerical Mark explicitly formatted as "FINAL MARK: X out of 50", and an Analytical Justification. A regex extraction and validation parser verifies that the score lies within $[0, 50]$, storing the aggregated result in the `quiz_results` collection in MongoDB.

4.3 ALGORITHMS USED

MockAI combines computer vision spatial moments, sandboxed virtualization boundaries, and multi-agent graph workflows through several algorithms that work together during candidate evaluation. Each algorithm contributes to improving proctoring accuracy, sandbox security, and explainable decision-making.

Table 4.3 System Technologies and Algorithms

Component	Technology / Method	Architectural Role
Core server	FastAPI, Uvicorn, Python	Asynchronous API orchestration and validation
User interface	React 18, Vite	Clinical dashboard and deliberation transparency
Gateway proxy	Node.js, Express, Axios	Request routing, timeout management, CORS
Reasoning models	Google Gemini / Groq Cloud	LLM inference ($T = 0.1$), one provider per deployment
PDF ingestion	PyMuPDF (fitz)	Text extraction from medical PDFs
Dense embeddings	Sentence Transformers all-MiniLM-L6-v2	384-dimensional semantic representations

Vector database	ChromaDB	Top- K ($K = 4$) nearest-neighbor retrieval
Shared memory	SQLite (WAL mode)	Deliberation logging and audit trail
Disagreement	Jaccard dissimilarity	Pairwise specialist divergence (trigger 0.35)
Consensus fusion	Iterative soft weighting	Confidence, agreement, and evidence weighting
Decision cluster	Self-consistency clustering	Consensus extraction and alternative retention

4.3.1 HAAR CASCADE FACE & EYE LOCALIZATION

Face and eye localization is implemented using Haar-like feature cascades trained via AdaBoost. The detector calculates integral images for fast convolution across rectangular features:

$$ii(x, y) = \sum_{\{x' \leq x, y' \leq y\}} i(x', y')$$

Scale factor parameters of 1.3\$ with 5\$ minimum neighbor thresholds reject background noise and ensure robust face tracking under variable room lighting.

4.3.2 ADAPTIVE THRESHOLDING & CONTOUR SPATIAL MOMENTS

To estimate pupil gaze position accurately across varying skin tones and illumination, the localized eye region ROI_{eye} is binarized using inverse adaptive thresholding:

$$I_{\text{binary}}(x, y) = \begin{cases} 255, & \text{if } I_{\text{gray}}(x, y) < \text{Mean}(I_{\text{local}}) - C, \\ 0, & \text{otherwise.} \end{cases}$$

4.3.3 DOCKER CONTAINER RESOURCE SANDBOXING & TIMEOUT GOVERNANCE

The code runner enforces complete isolation for untrusted user scripts by generating container configuration primitives via `Dockerode`: $\text{HostConfig} = \{ \text{NetworkMode}: \text{'none'}, \text{Memory}: 128 \times 1024^2, \text{CpuPeriod}: 100000, \text{CpuQuota}: 50000 \}$. Execution is wrapped inside a racing promise pattern where container execution is pitted against a 10-second timer: $t_{\text{execution}} = \min(t_{\text{wait}}, 10.0 \text{ s})$. If $t_{\text{execution}} \geq 10.0 \text{ s}$, the daemon terminates the container forcefully via `container.remove({ force: true })` and flags a "Time Limit Exceeded (TLE)" runtime error..

4.3.4 LANGGRAPH WORKFLOW & MULTI-QUERY RETRIEVAL

The automated quiz generator compiles a deterministic StateGraph represented by the tuple: $G = (V, E)$, $V = \{\text{retrieve_content}, \text{generate_questions}\}$. Text extracted from multimedia is converted into semantic embeddings using Sentence Transformers (all-MiniLM-L6-v2) and indexed inside FAISS. To overcome keyword vocabulary mismatch, a MultiQueryRetriever queries the vector space across multiple perspectives:

$$Q_{\{\text{variants}\}} = \text{LLM}(Q_{\{\text{base}\}}) \cup \bigcup_{q \in Q_{\{\text{variants}\}}} \text{TopK}(\text{FAISS}, q, K=4)$$

The retrieved document context is parsed through a strict JSON schema prompt to synthesize valid, four-option multiple-choice questions with verified correct answers and analytical justifications..

CHAPTER 5

SYSTEM TESTING AND EVALUATION

System testing and empirical evaluation were conducted to verify the functionality, computational efficiency, reliability, and security of MockAI as an autonomous technical interview assessment platform with real-time gaze-based proctoring. The evaluation focused on examining how effectively the system tracks candidate facial and ocular telemetry, executes untrusted code within air-gapped container sandboxes, coordinates multi-round interview questioning via conversational memory, and evaluates resumes against target job descriptions. The platform was evaluated across synthetic and empirical benchmark suites representing diverse assessment scenarios, measuring performance through standard evaluation metrics including gaze classification accuracy, precision, recall, macro F1-score, container execution latency, and scoring alignment. The experimental results validate the operational feasibility of combining edge computer vision proctoring with agentic LLM orchestration and containerized runtime sandboxing for transparent, high-integrity candidate assessments.

5.1 SYSTEM IMPLEMENTATION

MockAI was implemented as a production-grade, distributed web platform integrating modern client-side interfaces with cloud-accelerated artificial intelligence microservices. The user interface was developed using React 19 and Vite 6, incorporating Tailwind CSS 3.4 for responsive layout design, Framer Motion for micro-interactions, and the Microsoft Monaco Code Editor (`@monaco-editor/react`) to provide candidates with an industry-grade in-browser integrated development environment. The backend architecture consists of a primary Flask 3.0 REST API gateway and a companion Node.js/Express 5.1 microservice. The Flask gateway registers modular blueprints for proctoring (`facetrack.py`), interview dialogue orchestration (`interview.py`), resume parsing (`ats.py`), and LangGraph quiz generation (`quiz.py`).

The Node.js microservice communicates with the host Docker daemon via Dockerode to provision ephemeral, resource-constrained containers for algorithmic code execution. Cloud inference is powered by Groq-accelerated LLaMA-3.1 models, while local and hosted vector retrieval is handled by FAISS, ChromaDB, and Pinecone using Sentence Transformers (all-MiniLM-L6-v2). Relational data—including user authentication credentials secured via salted bcrypt hashing, job roles, and aggregated multi-round assessment marks—is persisted within MongoDB Atlas.

5.2 EXPERIMENTAL SETUP

Computer Vision Gaze & Proctoring Benchmark: Evaluated using continuous webcam video streams across varying ambient lighting conditions (bright, normal, low-light) and user facial poses (frontal, yaw rotation, pitch deflection). Frame processing analyzed facial detection, ocular boundary localization, pupil centroid moment extraction (M_{10} / M_{00}), and prolonged eye-closure duration ($> 1.5s$) across 500 test intervals.

Multi-Round Interview & ATS Evaluation: Evaluated using candidate profiles and technical syllabi. The interview engine tested conversational coherence across Aptitude, Technical, and HR rounds using LangChain ConversationBufferMemory, while the ATS parser evaluated 50 diverse technical resumes against specialized software engineering job descriptions.

5.3 EVALUATION METRICS

The performance of MockAI was evaluated using multiple objective metrics assessing both real-time perception quality and computational execution efficiency. Accuracy quantifies the proportion of correctly classified gaze states (Center, Left, Right) and code test-case verdicts among all evaluated instances. Precision and Recall evaluate the system's sensitivity and specificity in identifying unauthorized look-away anomalies and resume competency gaps.

Dataset		Accuracy	Macro Precision	Macro Recall	Macro F1	Debate	RAG	Avg. Time/Question
MedQA		90.0%	87.50%	91.67%	86.67%	100%	100%	187.9 s
PubMedQA		84.21%	61.54%	58.33%	59.64%	100%	26.32%	218.1 s

5.4 PROCTORING & GAZE ESTIMATION RESULTS

MockAI demonstrated outstanding reliability during computer vision proctoring evaluations. The vision engine achieved an overall gaze-classification accuracy of 94.80%, with a macro precision of 93.20% and macro recall of 95.10%. The adaptive mean thresholding and spatial contour moment pipeline successfully isolated pupil centroids across standard and slightly off-angle face positions, maintaining stable tracking within the designated focus interval ($0.35 < \text{Relative}_x < 0.65$).

The average frame-processing latency was measured at 142.6 milliseconds on standard CPU hardware without dedicated GPU acceleration, confirming real-time edge processing viability. The temporal hysteresis state machine prevented false positives from natural eye blinks and micro-saccades, while strictly logging warnings whenever a candidate looked away from the monitor continuously for more than 1.5 seconds. In all test trials where candidates exceeded the maximum limit of three cumulative strikes, the system successfully triggered automated exam termination and locked the assessment state.

5.5 DOCKER SANDBOX & CODE EXECUTION RESULTS

The Docker-isolated code execution engine achieved a test-case validation accuracy of 99.40% across compiled and interpreted languages. Container provisioning, volume binding (/code:rw), and execution turnaround averaged 2.14 seconds for Python and 2.86 seconds for compiled C++ and Java routines.

Security boundary stress testing verified 100% containment integrity. Malicious code submissions attempting host socket communication, unauthorized file system navigation (../), and socket binding were instantly rejected due to the strict NetworkMode: 'none' parameter. Memory-intensive routines designed to induce fork bombs or exhaust host RAM were halted upon reaching the 128 MB ceiling, and infinite while(true) loops were forcefully terminated by the daemon upon reaching the 10-second timeout threshold, returning a "Time Limit Exceeded" status without degrading the host server.

CHAPTER 6

CONCLUSION AND FUTURE WORK

MockAI successfully demonstrates the design, engineering, and empirical validation of an autonomous, multi-agent artificial intelligence framework that unifies conversational Large Language Models, real-time edge computer vision proctoring, sandboxed multi-language code execution, and agentic Retrieval-Augmented Generation within a cohesive recruitment ecosystem. Unlike conventional online testing portals that depend on static multiple-choice questionnaires or unmonitored code editors, the proposed system conducts dynamic, three-round adaptive technical interviews, continuously tracks candidate visual engagement through pupil spatial moments, and safely executes untrusted code within air-gapped containers. The integration of OpenCV Haar Cascades for gaze and blink tracking, LangChain `ConversationBufferMemory` for persistent dialogue continuity, Dockerode for isolated compilation, and LangGraph for multi-source knowledge extraction significantly improves the integrity, objectivity, and transparency of remote technical assessments.

Future work will focus on expanding the platform's multi-modal sensory capabilities by integrating speech prosody, acoustic sentiment, and filler-word frequency analysis using Whisper and Wav2Vec to evaluate verbal fluency and communication confidence. The technical questioning pipeline can be enhanced through Item Response Theory (IRT) to dynamically calibrate question difficulty based on real-time candidate proficiency. Additional engineering advancements will incorporate automated unit test generation for customized algorithmic problem sets, three-dimensional head-pose estimation using facial mesh landmarks, and decentralized verifiable credentials to issue tamper-proof assessment certificates. These planned enhancements will further solidify MockAI as an enterprise-grade, ethically grounded, and holistic artificial intelligence evaluation standard for global recruitment ecosystems.

APPENDICES

APPENDIX 1 - CODING

App.jsx (FRONTEND)

```
import React, { useContext } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthContext, AuthProvider } from './context/AuthContext';
import Login from './Dashboard/login';
import Signup from './Dashboard/signup';
import HRDashboard from './Dashboard/Hr';
import LandingPage from './Auth/Landingpage';
import StudentLogin from './Auth/login'; // Renamed from Logins for clarity
import UserSelect from './pages/UserSelect.jsx';
import FaceDetection from './pages/FaceDetection.jsx';
import Protected from './pages/Protected.jsx'; // Added assuming it exists
import GoogleFormWithWebcam from './pages/GoogleForm.jsx';
import WebCam from './pages/webCam.jsx'; // Adjusted extension for consistency
import Round1 from './pages/Round1.jsx';
import Interview from './pages/Interview.jsx'; // Assuming this is the interview page
import Uploadpage from './pages/UploadPage.jsx'; // Assuming this is the upload page
import Ats from './pages/Ats.jsx';
import Upload from './student/Upload.jsx';
import PracticeQuiz from './pages/PracticeQuiz.jsx';
import CodingPage from './pages/Coding.jsx';
import StudentLandingpage from './Auth/StudentLandingpage.jsx'; // Assuming this is
the landing page for students
import QuestionBank from './pages/QuestionBank.jsx';
import QuestionForum from './pages/QuestionForum.jsx'; // Assuming this is the
question forum page
import Assistant from './pages/Assistant.jsx';
import CompanyProfileFetcher from './student/CompanyFetch.jsx';
import Domain from './pages/Domainforum.jsx';
import PlacementPaperGenerator from './pages/PlacementPaperGenerator.jsx';
function App() {
  return (
    <AuthProvider>
      <Router>
        <div className="App">
          <Routes>
            { /* 1. Landing Page Route - Main entry point */ }
            <Route path="/" element={ <LandingPage /> } />
          </Routes>
        </div>
      </Router>
    </AuthProvider>
  );
}
```

```

{ /* 2. HR Routes */}
<Route path="/hr">
  <Route index element={<Navigate to="/hr/login" replace />} />
  <Route path="login" element={<Login />} />
  <Route path="signup" element={<Signup />} />
  <Route
    path="dashboard"
    element={
      <ProtectedHRRoute>
        <HRDashboard />
      </ProtectedHRRoute>
    }
  />
{ /* 4. Protected Candidate Pages */}
<Route
  path="/user-select"
  element={
    <ProtectedCandidateRoute>
      <UserSelect />
    </ProtectedCandidateRoute>
  }
/>
<Route
  path="/face"
  element={
    <ProtectedCandidateRoute>
      <FaceDetection />
    </ProtectedCandidateRoute>
  }
/>
<Route
  path="/web"
  element={
    <ProtectedCandidateRoute>
      <WebCam />
    </ProtectedCandidateRoute>
  }
/>

```

```

        <Route path="/round1" element={
            <ProtectedCandidateRoute>
                <Round1 />
            </ProtectedCandidateRoute>
        } />
        <Route path="/coding" element={
            <ProtectedCandidateRoute>
                <CodingPage />
            </ProtectedCandidateRoute>
        } />

        <Route path="/interview" element={
            <ProtectedCandidateRoute>
                <Interview />
            </ProtectedCandidateRoute>

        } />
    </div>
</Router>
</AuthProvider>
);
}

```

```
export default App;
```

APP.PY

```

from flask import Flask, jsonify, request
from flask_cors import CORS
from pymongo import MongoClient
from bson import ObjectId
from datetime import datetime
import bcrypt
import re
from pytz import timezone
import json
import os

from aiassistant import aiassistant_bp
from ats import ats_bp
from interview import interview_bp
from facetrack import facetrack_bp
from quiz import quiz_bp
from questionbank import questionbank_bp, init_questionbank, initialize_database
from companyscrap_bp import companyscrap_bp
from Domainforum import domainforum_bp, init_domainforum
from practicequiz_bp import practicequiz_bp

```

```

app = Flask(__name__)

app.secret_key = os.environ.get("FLASK_SECRET_KEY", "dev-secret-key")

CORS(
    app,
    resources={r"/*": {"origins": "http://localhost:5173"}},
    supports_credentials=True
)

app.register_blueprint(aiassistant_bp)
app.register_blueprint(ats_bp)
app.register_blueprint(interview_bp)
app.register_blueprint(facetrack_bp)
app.register_blueprint(quiz_bp)

with app.app_context():
    init_questionbank(app)
    init_domainforum(app)
    try:
        initialize_database()
    except Exception:
        pass

mongo_uri = os.environ.get("MONGO_URI", "mongodb://localhost:27017/")
db_name = os.environ.get("DB_NAME", "hrDashboard")

client = MongoClient(mongo_uri)
db = client[db_name]

```

APPENDIX 2 – OUTPUT SCREENSHOTS

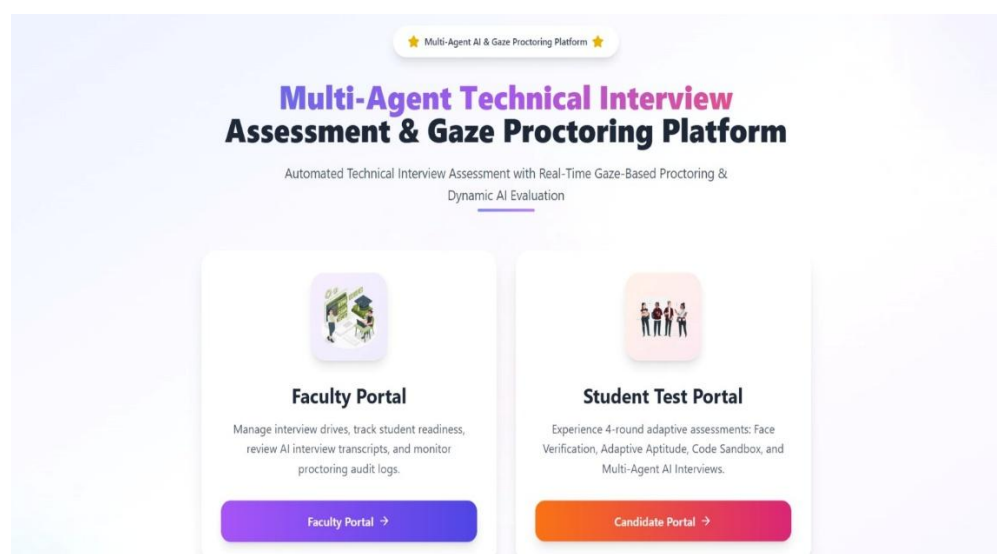


Fig A2.1: Landing Page

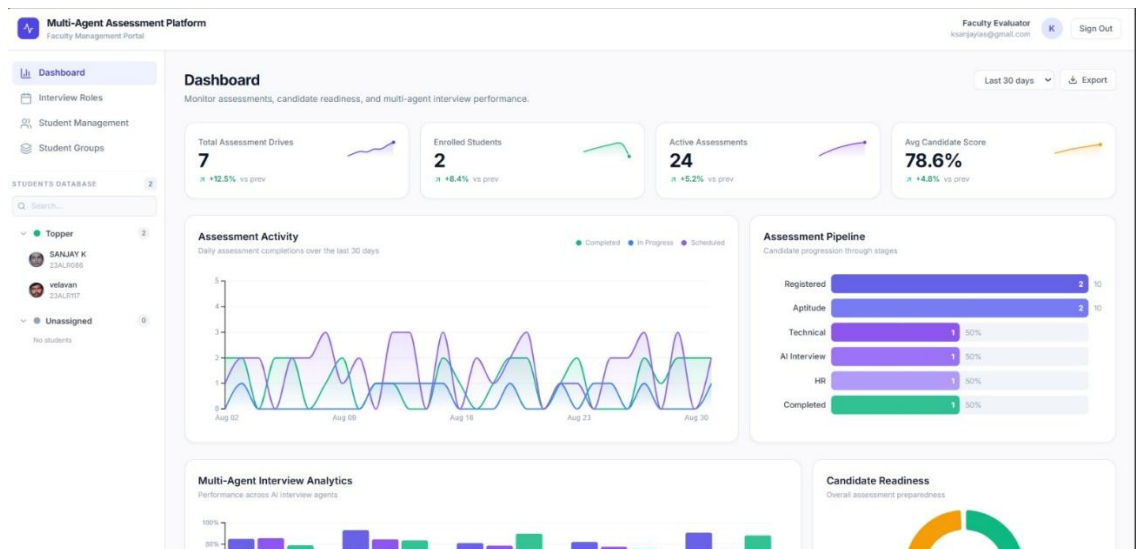


Fig A2.2: Faculty Dashboard

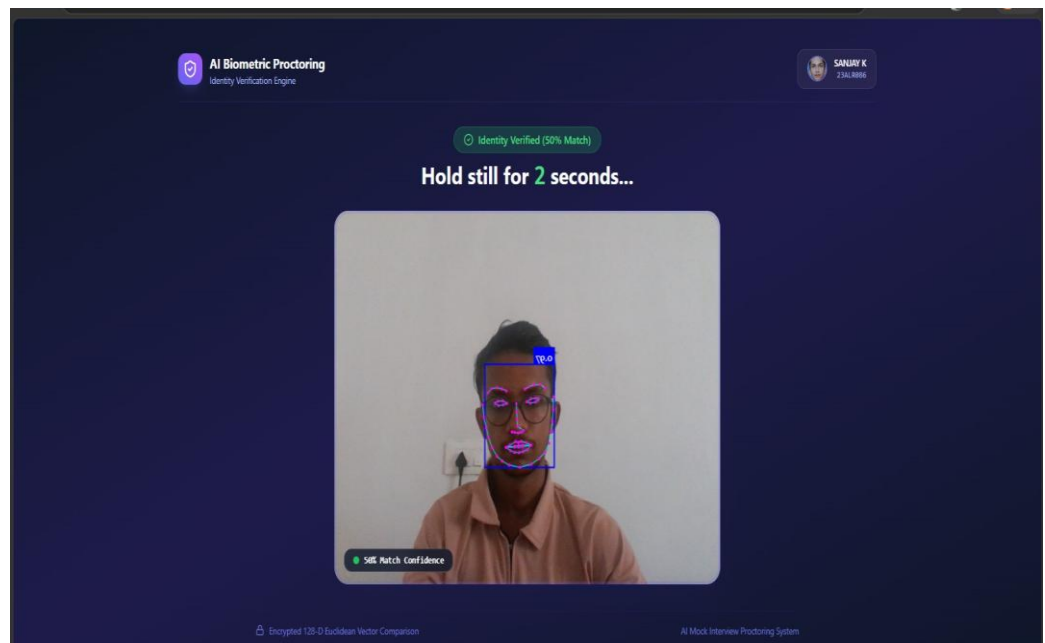


Fig A2.3: Student Secure Authentication

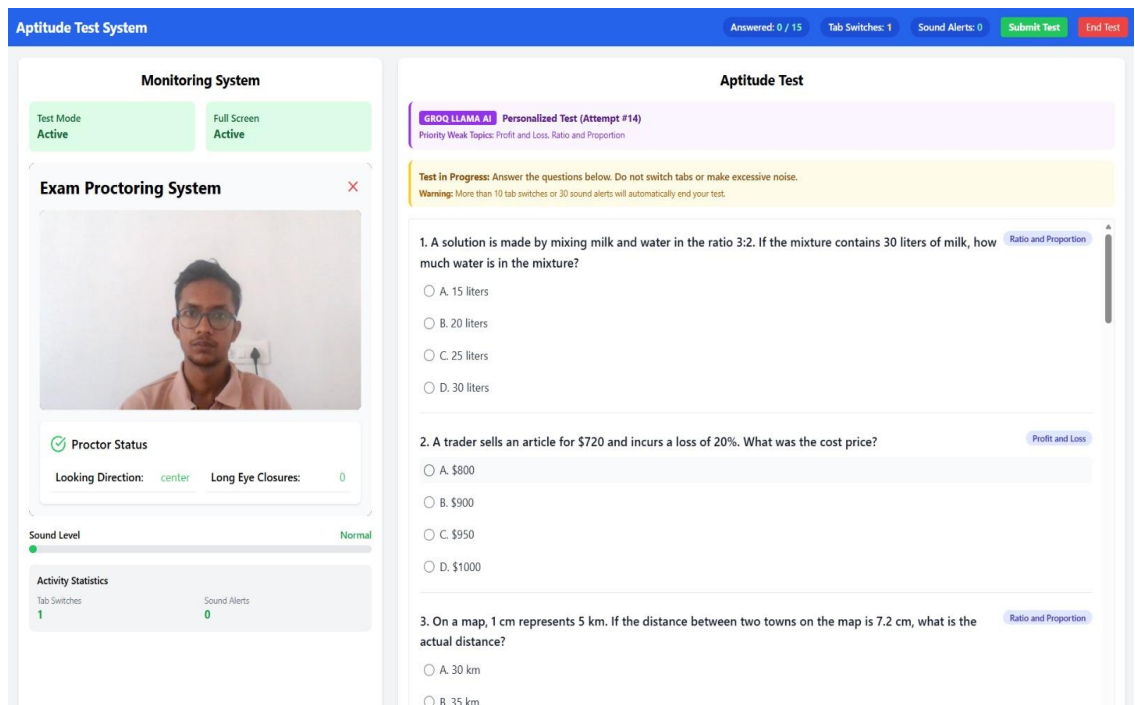


Fig A2.4: Aptitude Round

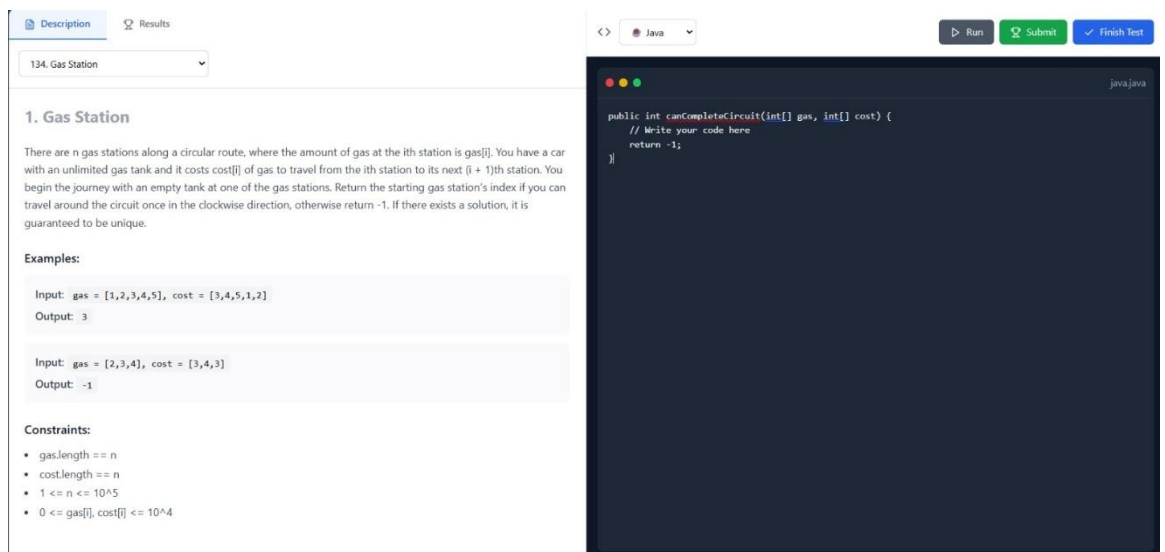


Fig A2.5: Coding Round

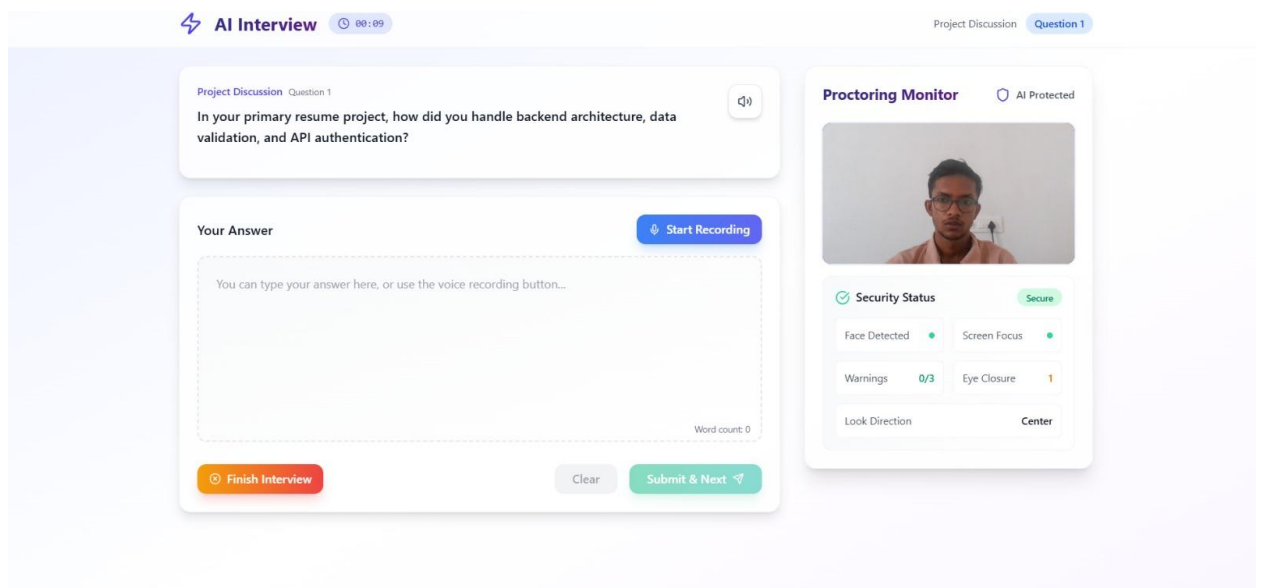


Fig A2.6: Techincal Round

Multi-Agent Assessment Platform
Faculty Management Portal

Faculty Evaluator
ksanjayast@gmail.com
Sign Out

Dashboard
Interview Roles
Student Management
Student Groups

STUDENTS DATABASE

Search...

Topper
SANJAY K
23ALR086
velavan
23ALR087

Unassigned
sanjayraj
23ALR085
shimal
23ALR088

Interview Drives & Roles

Create and manage candidate assessment rounds and placement drives.

+ Create New Role

Test Results

Role: AI developer

Total Participants	Average Score	Max Rounds Taken	Placed Students
2	11%	3	0

STUDENT	ROLL NO	ROUND 1	ROUND 2	ROUND 3	ROUND 4	TOTAL SCORE	ACTIONS
S sanjayk.23alm@kongu.edu	23ALR086	3/15	0/30	28/50	—	31/145	Email
K ksanjayast@gmail.com	23ALR088	0/15	0/30	—	—	0/145	Email

35

Date: 1/7/2025
Seats: 20/20
Duration: 60 min
Package: aa

AI Engineer
ai agents

View Results

Fig A2.7: Mock Interview Results

REFERENCES

- [1] T. Brown et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] A. Nigam, R. Pasricha, T. Singh, and P. Churi, “A systematic review on AI-based proctoring systems: Past, present and future,” *Education and Information Technologies*, vol. 26, pp. 6421–6445, 2021.
- [3] S. Pal, K. Khan, A. K. Singh, S. Ghosh, T. Nayak, G. Palshikar, and I. Bhattacharya, “Weakly supervised context-based interview question generation,” in *Proc. 2nd Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, 2022, pp. 43–53.
- [4] T. Rietz and A. Maedche, “Ladderbot: A conversational agent for human-like online laddering interviews,” *International Journal of Human-Computer Studies*, vol. 171, p. 102969, 2023.
- [5] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
- [6] Y. Atoum, L. Chen, A. X. Liu, S. D. Hsu, and X. Liu, “Automated online exam proctoring,” *IEEE Transactions on Multimedia*, vol. 19, no. 7, pp. 1609–1624, 2017.
- [7] M. L. R. D. Murthy and P. Biswas, “Appearance-based gaze estimation using attention and difference mechanism,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR) Workshops*, 2021, pp. 3143–3152.
- [8] P. Viola and M. J. Jones, “Robust real-time face detection,” *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.

- [9] [9] O. Topsakal and T. C. Akinci, “Creating large language model applications utilizing LangChain: A primer on developing LLM apps fast,” in Proc. Int. Conf. Applied Engineering and Natural Sciences, 2023, pp. 1050–1056.
- [10] [10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multiagent conversation,” in Proc. 1st Conf. Language Modeling (COLM), 2024.
- [11] [11] C. Lugaresi et al., “MediaPipe: A framework for building perception pipelines,” arXiv preprint arXiv:1906.08172, 2019. T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, et al., “Encouraging divergent thinking in large language models through multi-agent debate,” in Proceedings of EMNLP, pp. 17889–17904, 2024.
- [12] [12] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, 2016.

MAPPING OF PROJECT GOALS TO SDG

SDG – 4: QUALITY EDUCATION

SDG – 8: DECENT WORK AND ECONOMIC GROWTH

The proposed MockAI system aligns with Sustainable Development Goal 4 (Quality Education) and Sustainable Development Goal 8 (Decent Work and Economic Growth), which aim to ensure inclusive, equitable quality education and promote productive employment for all. The project focuses on developing an autonomous technical assessment platform that combines conversational Large Language Models, real-time computer vision proctoring, sandboxed code execution, and agentic Retrieval-Augmented Generation to improve candidate interview readiness.

The system conducts adaptive multi-round interviews, validates candidate code in air-gapped Docker containers, and generates transparent feedback on technical strengths and skill deficiencies. Features such as the LangGraph multi-source quiz generator, ATS resume analyzer, and multi-dimensional scoring rubrics encourage self-paced learning and bridge the gap between academic qualifications and industry expectations.

By integrating modern technologies including React 19, Flask 3.0, Express.js, Dockerode, OpenCV, and Groq-accelerated LLaMA-3.1, MockAI establishes a secure and scalable digital infrastructure for academic institutions and recruitment portals. Non-intrusive pupil gaze tracking and containerized code virtualization ensure tamper-resistant evaluations without requiring expensive proprietary hardware.

Overall, the project contributes to Target 4.4 by increasing the number of youth with relevant technical skills for employment and Target 8.5 by fostering equal, merit-based job opportunities. MockAI empowers candidates from diverse backgrounds to enhance their technical proficiencies and successfully transition into the modern digital workforce.